



Polymorphism

What will be covered

- Polymorphism basics
- Static and dynamic binding
- class Object
- Methods of class Object
- Covariant return type
- Annotation - @override
- Polymorphic variables
- Runtime polymorphism

- It is one of the core concepts of object-oriented programming (OOP).
- It is the ability of an object to take many forms.
- There are two types of polymorphism in java: compile-time polymorphism and runtime polymorphism. We can perform polymorphism in java by method overloading and method overriding.
- If you overload a static method in java, it is the example of compile time polymorphism
- Polymorphism allows us to create consistent code



Static Binding

The binding which can be resolved at compile time by compiler is known as static or early binding.

Static binding is based on type of reference on which the method is called

Binding of all the static, private and final methods is done at compile-time.

Static binding is better performance wise (no extra overhead is required).

Dynamic Binding

In Dynamic binding compiler doesn't decide the method to be called. Decision will be taken at runtime

Dynamic binding is based on type of object referred by the reference on which the method is called

Binding of all instance method except private and final methods is done late at runtime

Dynamic binding is slower as compared to static binding



- **Object** class is present in **java.lang** package.
- Every class in Java is directly or indirectly derived from the **Object** class.
- If a Class does not extend any other class then it is direct child class of **Object** and if extends other class then it is an indirectly derived.
- Therefore the Object class methods are available to all Java classes. Hence Object class acts as a root of inheritance hierarchy in any Java Program.
- The Object class provides some common behaviors to all the objects such as object can be compared, object can be cloned, object can be notified etc.

- **toString()** : **toString()** provides String representation of an Object and used to convert an object to String.
- **hashCode()** : For every object, JVM generates a unique number which is hashCode. It returns distinct integers for distinct objects
- **equals(Object obj)** : Compares the given object to "this" object (the object on which the method is called). It gives a generic way to compare objects for equality. It is recommended to override **equals(Object obj)** method to get our own equality condition on Objects
- **getClass()** : Returns the class object of "this" object
- **finalize()** method : This method is called just before an object is garbage collected.
- **clone()** : It returns a new object that is exactly the same as this object.
- **wait(), notify(), notifyAll()** : They are related to Concurrency.

Co-variant return types

- The overriding method in the subclass has the same name, number and type of parameters, and return type as the method that it overrides.
- An overriding method can also return a subtype of the type returned by the overridden method. This subtype is called a *covariant return type*.

```
public class SuperClass {  
    public SuperClass getAnObject(){  
        return this;  
    }  
  
    public class SubClass extends SuperClass {  
        public SubClass getAnObject(){  
            return this;  
        }  
    }  
}
```

- We get a liberty to have more specific return types when overriding methods.
- Help in preventing run-time ClassCastException on returns

- A variable is called polymorphic if it refers to different values under different conditions.
- Object variables (instance variables) represent the behavior of polymorphic variables in Java. It is because object variables of a class can refer to objects of its class as well as objects of its subclasses.

```
Employee e ;  
e = new SalesManager();  
e = new Programmer();
```

- In the above example e is a reference of class Employee. It can refer to object of Employee as well as its subclasses (child) SalesManager and Programmer.

- Runtime polymorphism or Dynamic Method Dispatch is a process in which a call to an overridden method is resolved at runtime rather than compile-time.
- In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.


```
class Shape{  
    void draw(){System.out.println("drawing...");}  
}  
  
class Rectangle extends Shape{  
    void draw(){System.out.println("drawing rectangle...");}  
}  
  
class Circle extends Shape{  
    void draw(){System.out.println("drawing circle...");}  
}
```

- In the above example, when reference `s` refers to `Rectangle` instance, `draw()` method is called from class `Rectangle` where as when reference `s` refers to `Circle`, `draw()` method is called from class `Circle`.

Assignments

1. Override toString() method from class Object in class Person, Emp and Date. Use this method to display information about Emp from main() method.
 2. Override toString() method from class Object in class Address, Customer and RegCustomer. Use this method to display information about Customer from main() method.
 3. Override toString() method from class Object in class Point, ColorPoint. Use this method to display information about Point from main() method.
 4. Extend above person hierarchy further to create specialized types of Emp like SalesManager(target,incentive), Programmer(extra_hours,charges_per_hour) and Admin(bonus). Create array of all different types of employees and call toString() method for all Emp subclasses.
 5. Create method calSalary() in Emp and override this method in Emp hierarchy. Call calSalary() method for all employees and display the total salary. Display the outcome like
Salary of Emp having <empid> is <salary>
 6. Create subclasse from Book(bookid, title, price) class as EBook and PaperBook. In EBook give discount (as mentioned below) on the original cost and for PaperBook add delivery charges(as mentioned below). Add the method calculateCost() in all the classes.
if the cost is in the specified range, add following discount

 <=500 - 2%

 >500 and <=1000 - 5%

 >1000 and <=5000 - 7%

if the cost is in the specified range, add following delivery charges

 <=500 - 7%

 >500 and <=1000 - 5%

 >1000 and <=5000 - 2%
- Create array of 5 books. Accept type of the book and accordingly information about 5 books and call calCost() method

Assignments-1

1. Extend above person hierarchy further to create specialized types of Emp like SalesManager(target,incentive), Programmer(extra_hours,charges_per_hour) and Admin(bonus). Create method calSalary() in Emp and override this method in Emp hierarchy. Create array of all different types of employees and call their calSalary() method and display the

Assignments-1

1. Extend above person hierarchy further to create specialized types of Emp like SalesManager(target,incentive), Programmer(extra_hours,charges_per_hour) and Admin(bonus). Create method calSalary() in Emp and override this method in Emp hierarchy. Create array of all different types of employees and call their calSalary() method and display the total salary.

2. Create subclasse from Book class as EBook and PaperBook. In EBook give discount (as mentioned below) on the original cost and for PaperBook add delivery charges(as mentioned below). Add the method calculateCost() in all the classes. if the cost is in the specified range, add following discount

<=500 - 2%

>500 and <=1000 - 5%

>1000 and <=5000 - 7%

if the cost is in the specified range, add following delivery charges

<=500 - 7%

>500 and <=1000 - 5%

>1000 and <=5000 - 2%

Create array of 5 books. Accept type of the book and accordingly information about 5 books and call calCost() method.

Assignments-2

1. Override toString() method in Person, Emp and Date and check the output by printing Emp object.

2. Extend above person hierarchy further to create specialized types of Emp like SalesManager(target,incentive), Programmer(extra_hours,charges_per_hour) and Admin(bonus). Create method calSalary() in Emp and override this method in Emp hierarchy. Create array of all different types of employees and call their calSalary() method and display the total salary.

3. Create subclasse from Book class as EBook and PaperBook. In EBook give discount (as mentioned below) on the original cost and for PaperBook add delivery charges(as mentioned below). Add the method calculateCost() in all the classes. if the cost is in the specified range, add following discount

<=500 - 2%

>500 and <=1000 - 5%

>1000 and <=5000 - 7%

if the cost is in the specified range, add following delivery charges

<=500 - 7%

method in Emp hierarchy. Create array of all different types of employees and call their calSalary() method and display the total salary.

3. Create subclasse from Book class as EBook and PaperBook. In EBook give discount (as mentioned below) on the original cost and for PaperBook add delivery charges(as mentioned below). Add the method calculateCost() in all the classes.

if the cost is in the specified range, add following discount

≤ 500 - 2%

> 500 and ≤ 1000 - 5%

> 1000 and ≤ 5000 - 7%

if the cost is in the specified range, add following delivery charges

≤ 500 - 7%

> 500 and ≤ 1000 - 5%

> 1000 and ≤ 5000 - 2%

Create array of 5 books. Accept type of the book and accordingly information about 5 books and call calCost() method.

Important Questions for: Polymorphism

Interview Questions

Q1. Explain any concept of core java?

Q2. what is polymorphism and where you used in your project show me?

Q3. what is use of @autowired?

Q4. abstraction , where you used in your project show source code of your project?

Explain polymorphism?

Real world example and write a code ?

Write a interface and its implementing class and one method in it where you use any logic?

explain polymorphism in hindi ?

- rate yourself in Java

- what is polymorphism in Java

- what is overloading and overriding

What is method overloading and method overriding.



